컴퓨터 실험 – Ⅰ 뱅킹 시스템

학번: 202601895 이름: 위성현 분반: 01(오전)

프로젝트: TOY Banking System

1.과제 정의

고객의 계좌 정보를 TXT 파일에서 불러와 계좌별 잔액을 관리하는 프로그램. 기존 계좌 로그인, 신규 사용자 등록, 입금, 출금, 잔액조회와 작업 내역 조회 기능을 제공하고 변경된 계좌 정보를 파일에 저장한다.

2. 요구 정의

a. 최대 100개의 계좌를 구조체 배열로 관리한다.

b. 각 계좌는 계좌번호, 비밀번호, 이름, 잔액을 가진다.

c. 프로그램 시작 시 customer_info.txt에서 계좌 정보를 불러온다.

d. 기존 계좌번호를 확인하여 업무 메뉴로 들어가고, 출금·이체·잔액조회에서 비밀번호를 확인한다.

e. 입금과 출금은 로그인한 계좌의 잔액에 반영한다.

f. 신규 사용자는 이름, 계좌번호, 비밀번호를 입력하고 잔액은 0원으로 등록한다.

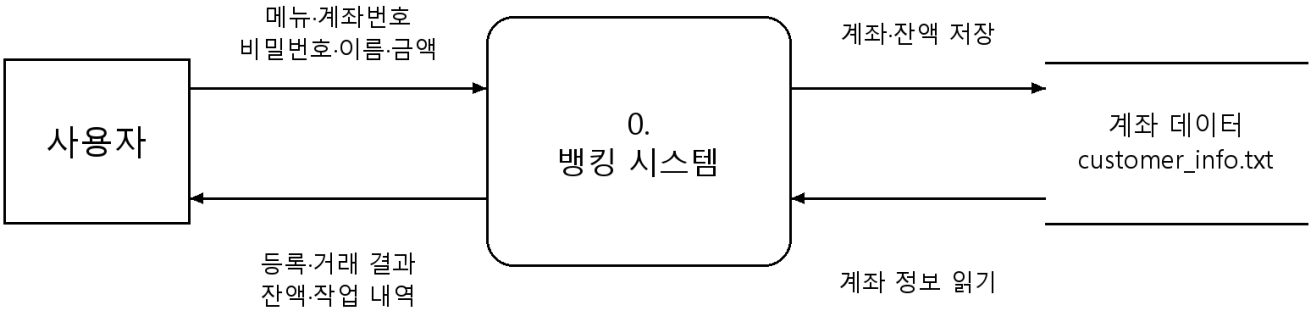
g. 계좌번호가 중복되면 등록을 취소하고 시작 메뉴로 돌아간다.

h. 변경된 계좌 정보를 저장하여 프로그램을 다시 실행해도 유지한다.

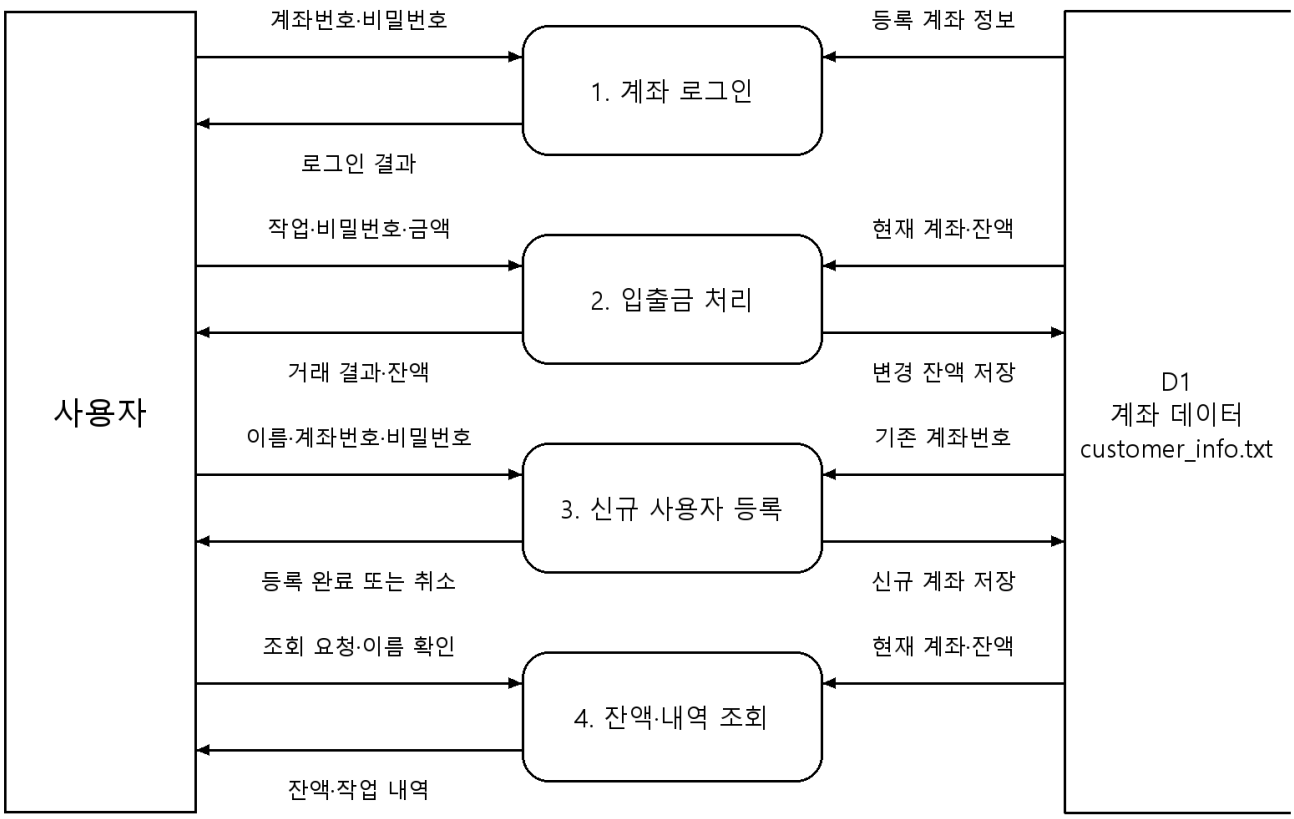
i. 내역 확인 시 이름을 확인하여 조회하고, 네·아니오 모두 시작 메뉴로 돌아간다.

3. 모델링

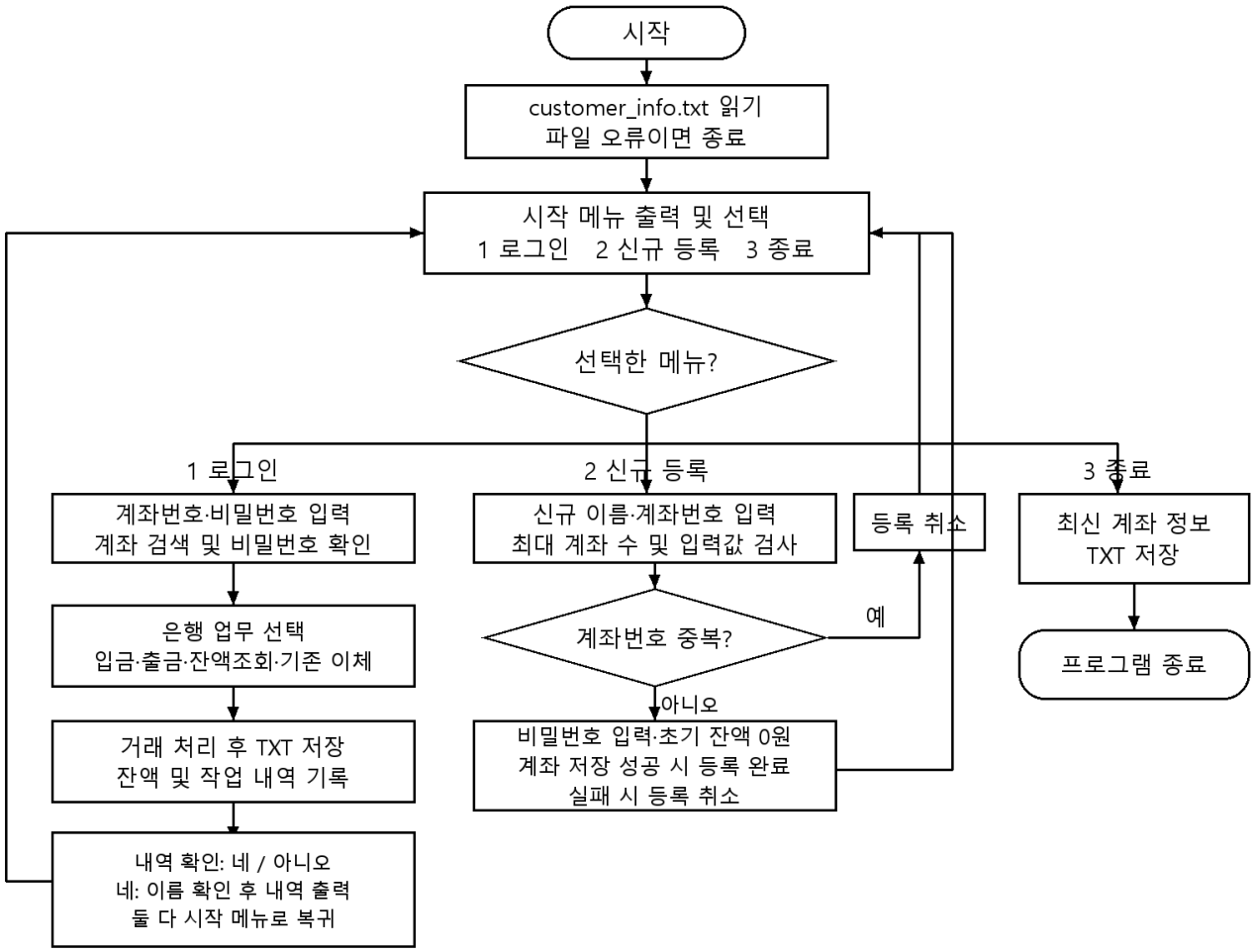
자료 흐름도 Level 0



자료 흐름도 Level 1



순서도


시작
customer_info.txt 읽기
파일 오류이면 종료
시작 메뉴 출력 및 선택
1 로그인 2 신규 등록 3 종료
선택한 메뉴?
1 로그인
2 신규 등록
3 종료
계좌번호·비밀번호 입력
계좌 검색 및 비밀번호 확인
은행 업무 선택
입금·출금·잔액조회·기존 이체
거래 처리 후 TXT 저장
잔액 및 작업 내역 기록
내역 확인: 네 / 아니오
네: 이름 확인 후 내역 출력
둘 다 시작 메뉴로 복귀
신규 이름·계좌번호 입력
최대 계좌 수 및 입력값 검사
계좌번호 중복?
예
아니오
비밀번호 입력·초기 잔액 0원
계좌 저장 성공 시 등록 완료
실패 시 등록 취소
등록 취소
최신 계좌 정보
TXT 저장
프로그램 종료

4. 테스트 케이스

※ 김백제·이온조·최쿠농의 초기 잔액은 10,000원·25,000원·100,000원이다. 테스트는 별도 파일에서 수행한다.

테스트 구분	입력 내용	초기 상태	예상 결과
파일 읽기	customer_info.txt 정상 파일	기존 3계좌	3계좌와 이름·잔액 불러오기
없는 파일	customer_info.txt 없이 실행	파일 없음	오류 처리 후 종료
입금	김백제 계좌에 5,000원 입금	10,000원	15,000원 반영
출금	김백제 계좌에서 3,000원 출금	10,000원	7,000원 반영
파일 저장	입금 후 TXT 내용 확인	10,000원	15,000원 저장
재실행	저장한 파일로 다시 잔액조회	15,000원 저장	15,000원 유지

5. 코드 구현(코딩)

banking.h

```
#ifndef BANKING_H
#define BANKING_H

#define N 20
#define MAX_ACCOUNTS 100

struct human
{
    long long ac_num; // 등록되어 있는 계좌번호
    short pw; // 등록되어 있는 비밀번호
    char name[N]; // 등록되어 있는 이름
    long long balance;
};

struct log
{
    char work[N]; // 작업 종류
    long long money; // 거래 금액
    long long balance; // 거래 후 잔액
};

// 계좌 검색
int find_account(struct human human_list[], int count, long long input_ac_num);

// 비밀번호 확인
int check_password(struct human human_list[], int account_index, short input_pw);

// 입금
void deposit(long long* balance, long long in_money);

// 출금
int withdraw(long long* balance, long long out_money);

// 이체
int transfer(long long* balance, long long out_money);

// 잔액조회
void show_balance(long long balance);

// 이름 확인
int check_name(struct human human_list[], int account_index, char input_name[]);

// 로그 저장
```

```
void save_log(
    struct log log_list[],
    int* log_count,
    char work[],
    long long money,
    long long balance
);

// 로그 출력
void show_log(struct log log_list[], int log_count);
// 계좌 파일 읽기 및 저장 (실패: -1, 빈 파일: 0)
int read_file(const char *file_name, struct human human_list[], int max_accounts);
int save_file(const char *file_name, const struct human human_list[], int account_count);

// 범위를 검사하는 입력 (성공: 1, 잘못된 입력: 0, 입력 종료: -1)
int read_text(char text[], int size);
int parse_number(const char text[], long long *number);
int read_number(long long *number);
int read_password(short *password);

#endif
```

main.c

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>
#include <limits.h>
#include "banking.h"

int main(void)
{
    struct human human_list[MAX_ACCOUNTS];

    struct log log_list[N];

    int log_count = 0;
    int account_count;
    int input_result;
    long long menu;
    const char *file_name = "customer_info.txt";
    int account_index;
    int check = 0;

    long long previous_balance;
    long long input_ac_num;
    long long in_money;
    long long out_money;

    char want_work[N];

    account_count = read_file(file_name, human_list, MAX_ACCOUNTS);
    if (account_count < 0)
    {
        printf("계좌 파일을 읽을 수 없거나 데이터 형식이 잘못되었습니다.₩n");
        return 1;
    }
    if (account_count == 0) printf("등록된 계좌가 없습니다. 신규 사용자를 등록하십시오.₩n");

start_menu:
    check = 0;
    log_count = 0;
    account_index = -1;
    while (1)
    {
        printf("========== BANKING SYSTEM ==========₩n₩n");
        printf("1. 기존 계좌 로그인₩n2. 신규 사용자 등록₩n3. 프로그램 종료₩n₩n");
        printf("원하시는 작업을 선택하십시오.:");
        input_result = read_number(&menu);
        if (input_result == -1) goto finish;
```

```
if (input_result != 1)
{
    printf("입력값이 잘못 되었습니다.₩n");
    continue;
}
if (menu == 3) goto finish;
if (menu == 2)
{
    struct human new_user;
    if (account_count >= MAX_ACCOUNTS)
    {
        printf("최대 계좌 개수를 초과하여 등록할 수 없습니다.₩n메인 메뉴로 돌아갑니다.₩n");
        continue;
    }
    printf("신규 사용자 이름을 입력하십시오.:");
    input_result = read_text(new_user.name, N);
    if (input_result == -1) goto finish;
    if (input_result != 1)
    {
        printf("이름이 잘못되었습니다. 사용자 등록이 취소되었습니다.₩n");
        continue;
    }
    printf("등록할 계좌번호를 입력하십시오.:");
    input_result = read_number(&new_user.ac_num);
    if (input_result == -1) goto finish;
    if (input_result != 1 || new_user.ac_num <= 0)
    {
        printf("계좌번호가 잘못되었습니다. 사용자 등록이 취소되었습니다.₩n");
        continue;
    }
    if (find_account(human_list, account_count, new_user.ac_num) != -1)
    {
        printf("이미 등록된 계좌번호입니다.₩n사용자 등록이 취소되었습니다.₩n₩n메인 메뉴로 돌아갑니다.₩n");
        continue;
    }
    printf("등록할 비밀번호를 입력하십시오.:");
    input_result = read_password(&new_user.pw);
    if (input_result == -1) goto finish;
    if (input_result != 1)
    {
        printf("비밀번호가 잘못되었습니다. 사용자 등록이 취소되었습니다.₩n");
        continue;
    }
    new_user.balance = 0;
    human_list[account_count] = new_user;
    if (save_file(file_name, human_list, account_count + 1) != 0)
    {
        printf("파일 저장에 실패하여 사용자 등록이 취소되었습니다.₩n메인 메뉴로 돌아갑니다.₩n");
        continue;
```

```
            }
            account_count++;
            printf("신규 사용자 등록이 완료되었습니다.₩n₩n계좌번호: %lld₩n이름: %s₩n초기 잔액: 0원₩n₩n메인 메뉴로 돌아갑니다.₩n", new_user.ac_num, new_user.name);
            continue;
        }
        if (menu != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }
        printf("등록하신 계좌번호를 입력하십시오.:");
        input_result = read_number(&input_ac_num);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }
        account_index = find_account(human_list, account_count, input_ac_num);
        if (account_index == -1)
        {
            printf("입력하신 계좌번호를 찾을 수 없습니다.₩n");
            continue;
        }
        break;
    }

    while (1)
    {
        printf("원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):");
        input_result = read_text(want_work, N);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }

        // 입금
        if (strcmp(want_work, "입금") == 0)
        {
            printf("입금할 금액을 입력하여 주십시오.:");
            input_result = read_number(&in_money);
            if (input_result == -1) goto finish;
            if (input_result != 1)
            {
                printf("입력값이 잘못 되었습니다.₩n");
                continue;
```

```
}

if (in_money <= 0 || in_money > LLONG_MAX - human_list[account_index].balance)
{
    printf("입금 금액이 올바르지 않습니다.₩n");
}

else
{
    previous_balance = human_list[account_index].balance;
    deposit(&human_list[account_index].balance, in_money);
    if (save_file(file_name, human_list, account_count) != 0)
    {
        human_list[account_index].balance = previous_balance;
        printf("파일 저장에 실패하여 거래가 취소되었습니다.₩n");
        continue;
    }

    printf("입금이 완료 되었습니다. 현재 잔액은 %lld원 입니다.₩n",
        human_list[account_index].balance);

    save_log(
        log_list,
        &log_count,
        "입금",
        in_money,
        human_list[account_index].balance
    );

    while (1)
    {
        char in_want_work[N];

        printf("작업내용을 확인 하시겠습니까?(네 또는 아니오):");
        input_result = read_text(in_want_work, N);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }

        if (strcmp(in_want_work, "네") == 0)
        {
            check = 1;
            break;
        }
```

```
            else if (strcmp(in_want_work, "아니오") == 0)
            {
                check = 2;
                break;
            }

            else
            {
                printf("입력값이 잘못 되었습니다.₩n");
            }
        }

        if (check != 0)
            break;
    }
}

// 출금
else if (strcmp(want_work, "출금") == 0)
{
    short input_pw;

    while (1)
    {
        printf("등록된 계좌의 비밀번호를 입력하십시오.:");
        input_result = read_password(&input_pw);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }

        if (check_password(human_list, account_index, input_pw) == 1)
        {
            printf("비밀번호가 올바르게 입력되었습니다.₩n");
            break;
        }

        else
        {
            printf("비밀번호가 틀립니다.₩n");
        }
    }

    printf("출금할 금액을 입력하여 주십시오.:");
```

```
input_result = read_number(&out_money);
if (input_result == -1) goto finish;
if (input_result != 1)
{
    printf("입력값이 잘못 되었습니다.₩n");
    continue;
}

previous_balance = human_list[account_index].balance;
if (withdraw(&human_list[account_index].balance, out_money) == 1)
{
    if (save_file(file_name, human_list, account_count) != 0)
    {
        human_list[account_index].balance = previous_balance;
        printf("파일 저장에 실패하여 거래가 취소되었습니다.₩n");
        continue;
    }
    printf("출금이 완료 되었습니다. 현재 잔액은 %lld원 입니다.₩n",
        human_list[account_index].balance);

    save_log(
        log_list,
        &log_count,
        "출금",
        out_money,
        human_list[account_index].balance
    );

    while (1)
    {
        char in_want_work[N];

        printf("작업내용을 확인 하시겠습니까?(네 또는 아니오):");
        input_result = read_text(in_want_work, N);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }

        if (strcmp(in_want_work, "네") == 0)
        {
            check = 1;
            break;
        }

        else if (strcmp(in_want_work, "아니오") == 0)
```

```
            {
                check = 2;
                break;
            }

            else
            {
                printf("입력값이 잘못 되었습니다.₩n");
            }
        }

        if (check != 0)
            break;
    }

    else
    {
        printf("출금 액수가 올바르지 않거나 잔액보다 큽니다.₩n");
    }
}

// 이체
else if (strcmp(want_work, "이체") == 0)
{
    short input_pw;

    while (1)
    {
        printf("등록된 계좌의 비밀번호를 입력하십시오.:");
        input_result = read_password(&input_pw);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }

        if (check_password(human_list, account_index, input_pw) == 1)
        {
            printf("비밀번호가 올바르게 입력되었습니다.₩n");
            break;
        }

        else
        {
            printf("비밀번호가 틀립니다.₩n");
        }
```

```
}

printf("현재 이체는 송금 계좌에서 차감만 하며 수취 계좌에 입금하지 않습니다.₩n");
printf("이체할 금액을 입력하여 주십시오.:");
input_result = read_number(&out_money);
if (input_result == -1) goto finish;
if (input_result != 1)
{
   printf("입력값이 잘못 되었습니다.₩n");
   continue;
}

previous_balance = human_list[account_index].balance;
if (transfer(&human_list[account_index].balance, out_money) == 1)
{
   if (save_file(file_name, human_list, account_count) != 0)
   {
      human_list[account_index].balance = previous_balance;
      printf("파일 저장에 실패하여 거래가 취소되었습니다.₩n");
      continue;
   }
   printf("이체 금액 차감이 완료 되었습니다. 현재 잔액은 %lld원 입니다.₩n",
      human_list[account_index].balance);

   save_log(
      log_list,
      &log_count,
      "이체",
      out_money,
      human_list[account_index].balance
   );

   while (1)
   {
      char in_want_work[N];

      printf("작업내용을 확인 하시겠습니까?(네 또는 아니오):");
      input_result = read_text(in_want_work, N);
      if (input_result == -1) goto finish;
      if (input_result != 1)
      {
         printf("입력값이 잘못 되었습니다.₩n");
         continue;
      }

      if (strcmp(in_want_work, "네") == 0)
      {
         check = 1;
```

```
            break;
        }

        else if (strcmp(in_want_work, "아니오") == 0)
        {
            check = 2;
            break;
        }

        else
        {
            printf("입력값이 잘못 되었습니다.₩n");
        }
    }

    if (check != 0)
        break;
}

else
{
    printf("이체 액수가 올바르지 않거나 잔액보다 큽니다.₩n");
}
}

// 잔액조회
else if (strcmp(want_work, "잔액조회") == 0)
{
    short input_pw;

    while (1)
    {
        printf("등록된 계좌의 비밀번호를 입력하십시오.:");
        input_result = read_password(&input_pw);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }

        if (check_password(human_list, account_index, input_pw) == 1)
        {
            printf("비밀번호가 올바르게 입력되었습니다.₩n");
            break;
        }
```

```
    else
    {
        printf("비밀번호가 틀립니다.₩n");
    }
}

show_balance(human_list[account_index].balance);

save_log(
    log_list,
    &log_count,
    "잔액조회",
    0,
    human_list[account_index].balance
);

while (1)
{
    char in_want_work[N];

    printf("작업내용을 확인 하시겠습니까?(네 또는 아니오):");
    input_result = read_text(in_want_work, N);
    if (input_result == -1) goto finish;
    if (input_result != 1)
    {
        printf("입력값이 잘못 되었습니다.₩n");
        continue;
    }

    if (strcmp(in_want_work, "네") == 0)
    {
        check = 1;
        break;
    }

    else if (strcmp(in_want_work, "아니오") == 0)
    {
        check = 2;
        break;
    }

    else
    {
        printf("입력값이 잘못 되었습니다.₩n");
    }
}
```

```
        if (check != 0)
            break;
    }

    // 종료
    else if (strcmp(want_work, "메인메뉴") == 0 || strcmp(want_work, "종료") == 0)
    {
        printf("시작 메뉴로 돌아갑니다.₩n₩n");
        goto start_menu;
    }

    else
    {
        printf("입력값이 잘못 되었습니다.₩n");
    }
}

// 작업 내역 확인
if (check == 1)
{
    char input_name[N];

    while (1)
    {
        printf("계좌 소유주의 이름을 입력하십시오.:");
        input_result = read_text(input_name, N);
        if (input_result == -1) goto finish;
        if (input_result != 1)
        {
            printf("입력값이 잘못 되었습니다.₩n");
            continue;
        }

        if (check_name(
            human_list,
            account_index,
            input_name
        ) == 1)
        {
            printf("실명 확인 되었습니다.₩n");
            break;
        }

        else
        {
            printf("귀하의 성명이 확인되지 않습니다.₩n");
        }
    }
```

```
        show_log(log_list, log_count);
    }
    printf("시작 메뉴로 돌아갑니다.₩n₩n");
    goto start_menu;

finish:
    if (save_file(file_name, human_list, account_count) != 0)
    {
        printf("계좌 파일 저장에 실패했습니다.₩n");
        return 1;
    }
    return 0;
}
```

bandking.c

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <errno.h>
#include <limits.h>
#include <ctype.h>
#include "banking.h"

// 계좌번호 검색
int find_account(struct human human_list[], int count, long long input_ac_num)
{
    int i;

    for (i = 0; i < count; i++)
    {
        if (input_ac_num == human_list[i].ac_num)
        {
            return i;
        }
    }

    return -1;
}

// 비밀번호 확인
int check_password(struct human human_list[], int account_index, short input_pw)
{
    if (input_pw == human_list[account_index].pw)
    {
        return 1;
    }

    return 0;
}

// 입금
void deposit(long long* balance, long long in_money)
{
    *balance = *balance + in_money;
}

// 출금
int withdraw(long long* balance, long long out_money)
{
    if (out_money > *balance)
```

```
        return 0;

    else if (out_money <= 0)
        return 0;

    else
    {
        *balance = *balance - out_money;
        return 1;
    }
}

// 이체
int transfer(long long* balance, long long out_money)
{
    if (out_money > *balance)
        return 0;

    else if (out_money <= 0)
        return 0;

    else
    {
        *balance = *balance - out_money;
        return 1;
    }
}

// 잔액조회
void show_balance(long long balance)
{
    printf("현재 귀하의 잔액은 %lld원 입니다.₩n", balance);
}

// 이름 확인
int check_name(struct human human_list[], int account_index, char input_name[])
{
    if (strcmp(human_list[account_index].name, input_name) == 0)
    {
        return 1;
    }

    return 0;
}

// 로그 저장
void save_log(
```

```
    struct log log_list[],
    int* log_count,
    char work[],
    long long money,
    long long balance
)
{
    if (*log_count < N)
    {
        strcpy(log_list[*log_count].work, work);

        log_list[*log_count].money = money;
        log_list[*log_count].balance = balance;

        (*log_count)++;
    }
}

// 로그 출력
void show_log(struct log log_list[], int log_count)
{
    int i;

    if (log_count == 0)
    {
        printf("확인할 작업 내역이 없습니다.₩n");
        return;
    }

    printf("₩n------ 작업 내역 ------₩n");

    for (i = 0; i < log_count; i++)
    {
        if (strcmp(log_list[i].work, "잔액조회") == 0)
        {
            printf("%d. %s / 현재 잔액 %lld원₩n",
                i + 1,
                log_list[i].work,
                log_list[i].balance);
        }

        else
        {
            printf("%d. %s / %lld원 / 거래 후 잔액 %lld원₩n",
                i + 1,
                log_list[i].work,
                log_list[i].money,
```

```
                log_list[i].balance);
        }
    }
}
// 한 줄 단위로 읽어 긴 입력의 나머지가 다음 입력에 섞이지 않도록 처리
static int read_line(FILE *file, char text[], int size)
{
    int ch, length = 0, invalid = 0;
    while ((ch = fgetc(file)) != '₩n' && ch != EOF)
    {
        if (ch == '₩0') invalid = 1;
        if (length < size - 1) text[length++] = (char)ch;
        else invalid = 1;
    }
    text[length] = '₩0';
    if (ferror(file)) return -1;
    if (ch == EOF && length == 0 && !invalid) return -1;
    if (invalid) return 0;
    if (length > 0 && text[length - 1] == '₩r') text[length - 1] = '₩0';
    return 1;
}

int read_text(char text[], int size)
{
    char line[256];
    char *start, *end;
    int result = read_line(stdin, line, sizeof(line));
    if (result != 1)
        return result;
    start = line;
    while (isspace((unsigned char)*start)) start++;
    end = start + strlen(start);
    while (end > start && isspace((unsigned char)end[-1])) end--;
    *end = '₩0';
    if (*start == '₩0' || strlen(start) >= (size_t)size)
        return 0;
    for (end = start; *end; end++)
        if (isspace((unsigned char)*end)) return 0;
    strcpy(text, start);
    return 1;
}

int parse_number(const char text[], long long *number)
{
    char *end;
    long long value;
    errno = 0;
    value = strtoll(text, &end, 10);
    if (errno == ERANGE || end == text || *end != '₩0')
```

```
        return 0;
    *number = value;
    return 1;
}

int read_number(long long *number)
{
    char text[64];
    int result = read_text(text, sizeof(text));
    return result == 1 ? parse_number(text, number) : result;
}

int read_password(short *password)
{
    long long number;
    int result = read_number(&number);
    if (result != 1) return result;
    if (number < 0 || number > SHRT_MAX) return 0;
    *password = (short)number;
    return 1;
}

int read_file(const char *file_name, struct human human_list[], int max_accounts)
{
    FILE *file;
    struct human loaded[MAX_ACCOUNTS];
    char line[256], ac_num[64], pw[64], name[256], balance[64], extra;
    long long password;
    int count = 0, result, i;
    if (file_name == NULL || human_list == NULL || max_accounts <= 0 || max_accounts > MAX_ACCOUNTS)
        return -1;
    file = fopen(file_name, "rb");
    if (file == NULL) return -1;
    while ((result = read_line(file, line, sizeof(line))) != -1)
    {
        if (result != 1 || count >= max_accounts ||
            sscanf(line, "%63s %63s %255s %63s %c", ac_num, pw, name, balance, &extra) != 4 ||
            strlen(name) >= N || !parse_number(ac_num, &loaded[count].ac_num) ||
            !parse_number(pw, &password) || !parse_number(balance, &loaded[count].balance) ||
            loaded[count].ac_num <= 0 || password < 0 || password > SHRT_MAX || loaded[count].balance < 0)
            goto read_error;
        for (i = 0; i < count; i++)
            if (loaded[i].ac_num == loaded[count].ac_num) goto read_error;
        loaded[count].pw = (short)password;
        strcpy(loaded[count].name, name);
        count++;
    }
    if (ferror(file)) goto read_error;
```

```
    if (fclose(file) != 0) return -1;
    memcpy(human_list, loaded, count * sizeof(loaded[0]));
    return count;

read_error:
    fclose(file);
    return -1;
}

int save_file(const char *file_name, const struct human human_list[], int account_count)
{
    FILE *file;
    char temporary[1024], backup[1024];
    int i, failed = 0;
    if (file_name == NULL || human_list == NULL || account_count < 0 || account_count > MAX_ACCOUNTS ||
        strlen(file_name) + 5 > sizeof(temporary)) return -1;
    sprintf(temporary, "%s.tmp", file_name);
    sprintf(backup, "%s.bak", file_name);

    file = fopen(file_name, "r");
    if (file == NULL) return -1;
    if (fclose(file) != 0) return -1;
    file = fopen(backup, "r");
    if (file != NULL)
    {
        fclose(file);
        printf("백업 파일이 남아 있습니다. %s를 확인하십시오.₩n", backup);
        return -1;
    }
    if (errno != ENOENT) return -1;
    file = fopen(temporary, "r");
    if (file != NULL)
    {
        fclose(file);
        printf("임시 파일이 남아 있습니다. %s를 확인하십시오.₩n", temporary);
        return -1;
    }
    if (errno != ENOENT) return -1;
    file = fopen(temporary, "w");
    if (file == NULL) return -1;
    for (i = 0; i < account_count; i++)
    {
        if (fprintf(file, "%lld %hd %s %lld₩n", human_list[i].ac_num,
            human_list[i].pw, human_list[i].name, human_list[i].balance) < 0)
        {
            failed = 1;
            break;
        }
```

```
    }
    if (fflush(file) != 0) failed = 1;
    if (fclose(file) != 0) failed = 1;
    if (failed || rename(file_name, backup) != 0)
    {
        remove(temporary);
        return -1;
    }
    if (rename(temporary, file_name) != 0)
    {
        if (rename(backup, file_name) != 0)
            printf("기존 파일 복구에 실패했습니다. %s와 %s를 보존하고 확인하십시오.₩n", backup, temporary);
        else remove(temporary);
        return -1;
    }
    if (remove(backup) != 0)
        printf("저장은 완료되었으나 백업 파일을 지우지 못했습니다. %s를 확인하십시오.₩n", backup);
    return 0;
}
```

customer_info.txt

100152351124 1524 김백제 10000
100144523657 1275 이온조 25000
100142566328 1685 최쿠농 100000

6. 실행 결과

테스트 항목	입력 및 조건	실제 실행 결과
파일 읽기	정상 TXT 읽기	계좌 3개와 한글 이름·잔액 확인 / 통과
없는 파일	TXT 없이 읽기	읽기 실패 처리, 계좌 배열 유지 / 통과
입금	10,000원에서 5,000원 입금	잔액 15,000원 / 통과
출금	10,000원에서 3,000원 출금	잔액 7,000원 / 통과
파일 저장	5,000원 입금 후 파일 확인	TXT 잔액 15,000원 / 통과
재실행	저장 후 새 프로세스 실행	잔액 15,000원 조회 / 통과

입금 결과

```
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):입금
입금할 금액을 입력하여 주십시오.:5000
입금이 완료 되었습니다. 현재 잔액은 15000원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):아니오
시작 메뉴로 돌아갑니다.
```

출금 결과

```
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):출금
등록된 계좌의 비밀번호를 입력하십시오.:1524
비밀번호가 올바르게 입력되었습니다.
출금할 금액을 입력하여 주십시오.:3000
출금이 완료 되었습니다. 현재 잔액은 7000원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):아니오
시작 메뉴로 돌아갑니다.
```

재실행 후 잔액 확인

```
등록하신 계좌번호를 입력하십시오.:100152351124
원하시는 작업을 입력해주세요( 입금, 출금, 이체, 잔액조회, 메인메뉴 ):잔액조회
등록된 계좌의 비밀번호를 입력하십시오.:1524
비밀번호가 올바르게 입력되었습니다.
현재 귀하의 잔액은 15000원 입니다.
작업내용을 확인 하시겠습니까?(네 또는 아니오):아니오
시작 메뉴로 돌아갑니다.
```

신규 사용자 등록 결과

```
신규 사용자 이름을 입력하십시오.:홍길동
등록할 계좌번호를 입력하십시오.:100199999999
등록할 비밀번호를 입력하십시오.:1234
신규 사용자 등록이 완료되었습니다.


계좌번호: 100199999999
이름: 홍길동
초기 잔액: 0원


메인 메뉴로 돌아갑니다.
```

계좌번호 중복 등록 결과

```
신규 사용자 이름을 입력하십시오.:홍길동
등록할 계좌번호를 입력하십시오.:100152351124
이미 등록된 계좌번호입니다.
사용자 등록이 취소되었습니다.


메인 메뉴로 돌아갑니다.
```

작업 내역과 처음 메뉴 복귀

```
계좌 소유주의 이름을 입력하십시오.:김백제
실명 확인 되었습니다.


------ 작업 내역 ------
1. 입금 / 5000원 / 거래 후 잔액 15000원
시작 메뉴로 돌아갑니다.


========== BANKING SYSTEM ==========


1. 기존 계좌 로그인
2. 신규 사용자 등록
3. 프로그램 종료
```

입금 후 customer_info.txt

```
100152351124 1524 김백제 15000
100144523657 1275 이온조 25000
100142566328 1685 최쿠농 100000
```